DOCUMENTO DE ENTREGA 2 - BANCO DE DADOS (VIEWS)

Projeto: Collision - Plataforma de Acompanhamento Escolar

📋 1. INTRODUÇÃO

Este documento apresenta a criação das **Views (Visões)** para o banco de dados Oracle do projeto Collision, conforme exigido na **Entrega 2** da disciplina de Banco de Dados.

O objetivo das views é:

- **Simplificar consultas complexas:** Evitar que a API (Node.js) precise fazer múltiplos `JOINs` pesados.
- **Garantir segurança (Row-Level Security):** Assegurar que um pai só veja os dados dos próprios filhos e que o professor veja apenas seus alunos.
- **Otimizar a geração de relatórios:** Fornecer dados prontos para o PDF e para o Excel.
- **Melhorar a performance:** O banco de dados processa a lógica de junção, entregando os dados mastigados para a aplicação.

🎯 2. METODOLOGIA

As views foram criadas pensando no fluxo do sistema e nos perfis de usuário:

1. **Administrador:** Precisa de uma fila de revisão e indicadores gerenciais.
2. **Professor:** Precisa ver seus registros e o status (se foi devolvido ou publicado).
3. **Pai/Responsável:** Só pode ver o que está publicado e vinculado ao seu CPF/Usuário.

A partir disso, foram criadas **5 views encadeadas**, onde a View 1 é a base para as demais.

📝 3. VIEWS CRIADAS E JUSTIFICATIVAS

🔹 VIEW 1: VW_ACOMPANHAMENTO_DETALHADO (View Base)

O que ela faz:

É a view mais importante. Ela junta 8 tabelas (`ACOMPANHAMENTO`, `ALUNO`, `TURMA`, `DISCIPLINA`, `PROFESSOR`, `USUARIO`, `BIMESTRE` e `TAG`) em uma única consulta. Ela traz **todos os status** (rascunho, enviado, publicado, etc.) e usa a função `LISTAGG` para agrupar as tags de cada aluno em uma única linha.

Justificativa:

Sem essa view, a API precisaria fazer 8 `JOINs` toda vez que o administrador abrisse o painel. Com ela, a consulta fica pronta e otimizada. Ela alimenta o painel de revisão e a exportação para Excel (RF10 e RF11).

Consulta de exemplo:

sql

```
SELECT nome_aluno, nome_turma, nome_disciplina, status
FROM VW_ACOMPANHAMENTO_DETALHADO
WHERE status IN ('ENVIADO_PARA_REVISAO', 'EM_REVISAO');
```

🔹 VIEW 2: VW_RELATORIO_PUBLICADO

O que ela faz:

Filtra a View Base trazendo **apenas** os registros com `status = 'PUBLICADO'`.

Justificativa:

Atende diretamente à regra de negócio do PDF (RF09): *"Somente registros publicados poderão ser visualizados pelo Pai/Responsável."* Ela é a base para a geração do relatório em PDF.

Consulta de exemplo:

sql

```
SELECT nome_aluno, numero_bimestre, nome_disciplina, media
FROM VW_RELATORIO_PUBLICADO
WHERE id_aluno = 1;
```

🔹 VIEW 3: VW_RELATORIO_RESPONSAVEL

O que ela faz:

Cruza os dados publicados do aluno com a tabela de responsáveis e verifica, através de um `LEFT JOIN` com a tabela de retorno, se o pai já tomou ciência (0 = pendente, 1 = ciente).

Justificativa:

Garante a segurança (Row-Level Security). Quando o pai logar no sistema, a API fará `SELECT * FROM VW_RELATORIO_RESPONSAVEL WHERE id_usuario_responsavel = ?`. Ele só verá os dados se estiver vinculado ao aluno (RF08 e RF12).

Consulta de exemplo:

sql

```sql
SELECT nome_aluno, media, ciencia
FROM VW_RELATORIO_RESPONSAVEL
WHERE id_usuario_responsavel = 3;
```

🔹 VIEW 4: VW_INDICADORES_TURMA

O que ela faz:

Agrupa os dados por turma, disciplina, professor e bimestre. Mostra quantos registros estão em rascunho, quantos publicados, a média geral, a maior/menor nota e quantos alunos estão abaixo de 6.0.

Justificativa:

Atende ao RF10 (Relatórios Gerenciais). O Administrador pode acompanhar a saúde da escola e o andamento do fluxo editorial em tempo real, sem rodar queries pesadas na aplicação.

Consulta de exemplo:

sql

```sql
SELECT nome_turma, nome_disciplina, media_publicada, qtd_ab
aixo_de_6
FROM VW_INDICADORES_TURMA
WHERE numero_bimestre = 1;
```

🔹 VIEW 5: VW_DISTRIBUICAO_TAGS

O que ela faz:

Mostra quantas vezes cada tag (ex: "Necessita reforço") foi usada por turma e bimestre, além de calcular a média dos alunos que receberam aquela tag.

Justificativa:

Permite que a coordenação identifique padrões pedagógicos (ex: "Muitos alunos com a tag 'Atenção' em Matemática no 1º bimestre") e tome ações direcionadas (RF10).

Consulta de exemplo:

sql

```sql
SELECT nome_turma, nome_tag, qtd_usos, media_dos_alunos
FROM VW_DISTRIBUICAO_TAGS
ORDER BY qtd_usos DESC;
```

💻 4. SCRIPT SQL COMPLETO (ORACLE)

sql

```sql
-- ============================================================
=============
-- COLLISION - ENTREGA 2 - BANCO DE DADOS (Oracle SQL)
-- VIEWs para relatórios, painel de revisão e indicadores g
erenciais
-- ============================================================
=============

-- 1. VIEW BASE (Junta as 8 tabelas)
CREATE OR REPLACE VIEW VW_ACOMPANHAMENTO_DETALHADO AS
SELECT
    a.id_acompanhamento,
    al.id_aluno,
    al.nome AS nome_aluno,
    al.matricula AS matricula_aluno,
    t.id_turma,
    t.nome AS nome_turma,
```

```
    v.ano_letivo,
    b.id_bimestre,
    b.numero AS numero_bimestre,
    d.id_disciplina,
    d.nome AS nome_disciplina,
    d.area AS area_disciplina,
    p.id_professor,
    up.id_usuario AS id_usuario_professor,
    up.nome AS nome_professor,
    a.media,
    a.descricao,
    a.status,
    (SELECT LISTAGG(tg.nome, ', ') WITHIN GROUP (ORDER BY t
g.nome)
       FROM ACOMPANHAMENTO_TAG atg
       JOIN TAG tg ON tg.id_tag = atg.id_tag
      WHERE atg.id_acompanhamento = a.id_acompanhamento
        AND atg.ativo = 1 AND tg.ativo = 1) AS tags,
    a.data_criacao,
    a.data_atualizacao
FROM ACOMPANHAMENTO a
JOIN ALUNO al ON al.id_aluno = a.id_aluno
JOIN VINCULO_PROFESSOR v ON v.id_vinculo = a.id_vinculo
JOIN TURMA t ON t.id_turma = v.id_turma
JOIN DISCIPLINA d ON d.id_disciplina = v.id_disciplina
JOIN PROFESSOR p ON p.id_professor = v.id_professor
JOIN USUARIO up ON up.id_usuario = p.id_usuario
JOIN BIMESTRE b ON b.id_bimestre = a.id_bimestre
WHERE a.ativo = 1;

-- 2. VIEW DE RELATÓRIO (Apenas Publicados - Base para o PD
F)
CREATE OR REPLACE VIEW VW_RELATORIO_PUBLICADO AS
SELECT
    id_acompanhamento, id_aluno, nome_aluno, matricula_alun
o,
    id_turma, nome_turma, ano_letivo, id_bimestre, numero_b
imestre,
```

```
    id_disciplina, nome_disciplina, area_disciplina, nome_p
rofessor,
    media, descricao, tags, data_atualizacao AS data_public
acao
FROM VW_ACOMPANHAMENTO_DETALHADO
WHERE status = 'PUBLICADO';

-- 3. VIEW DO RESPONSÁVEL (Cruzamento com o Retorno/Ciênci
a)
CREATE OR REPLACE VIEW VW_RELATORIO_RESPONSAVEL AS
SELECT
    resp.id_usuario AS id_usuario_responsavel,
    ar.id_responsavel,
    ar.parentesco,
    r.id_acompanhamento, r.id_aluno, r.nome_aluno, r.nome_t
urma,
    r.ano_letivo, r.numero_bimestre, r.nome_disciplina, r.n
ome_professor,
    r.media, r.descricao, r.tags, r.data_publicacao,
    NVL(ret.ciencia, 0) AS ciencia,
    ret.dt_ciencia
FROM VW_RELATORIO_PUBLICADO r
JOIN ALUNO_RESPONSAVEL ar ON ar.id_aluno = r.id_aluno AND a
r.ativo = 1
JOIN RESPONSAVEL resp ON resp.id_responsavel = ar.id_respon
savel AND resp.ativo = 1
LEFT JOIN (
    SELECT id_acompanhamento, id_responsavel,
           MAX(ciencia) AS ciencia, MAX(dt_ciencia) AS dt_c
iencia
    FROM RETORNO_RESPONSAVEL
    WHERE ativo = 1
    GROUP BY id_acompanhamento, id_responsavel
) ret ON ret.id_acompanhamento = r.id_acompanhamento
     AND ret.id_responsavel = ar.id_responsavel;

-- 4. VIEW DE INDICADORES GERENCIAIS (Dashboard Admin)
CREATE OR REPLACE VIEW VW_INDICADORES_TURMA AS
```

```sql
SELECT
    id_turma, nome_turma, ano_letivo, numero_bimestre,
    id_disciplina, nome_disciplina, id_professor, nome_prof
essor,
    COUNT(*) AS total_registros,
    COUNT(CASE WHEN status = 'RASCUNHO' THEN 1 END) AS qtd_
rascunho,
    COUNT(CASE WHEN status IN ('ENVIADO_PARA_REVISAO','EM_R
EVISAO') THEN 1 END) AS qtd_em_analise,
    COUNT(CASE WHEN status = 'DEVOLVIDO' THEN 1 END) AS qtd
_devolvidos,
    COUNT(CASE WHEN status = 'PUBLICADO' THEN 1 END) AS qtd
_publicados,
    COUNT(CASE WHEN status = 'CANCELADO' THEN 1 END) AS qtd
_cancelados,
    ROUND(AVG(CASE WHEN status = 'PUBLICADO' THEN media EN
D), 2) AS media_publicada,
    MIN(CASE WHEN status = 'PUBLICADO' THEN media END) AS m
enor_media,
    MAX(CASE WHEN status = 'PUBLICADO' THEN media END) AS m
aior_media,
    COUNT(CASE WHEN status = 'PUBLICADO' AND media < 6 THEN
1 END) AS qtd_abaixo_de_6
FROM VW_ACOMPANHAMENTO_DETALHADO
GROUP BY
    id_turma, nome_turma, ano_letivo, numero_bimestre,
    id_disciplina, nome_disciplina, id_professor, nome_prof
essor;

-- 5. VIEW DE TAGS (Relatório de distribuição)
CREATE OR REPLACE VIEW VW_DISTRIBUICAO_TAGS AS
SELECT
    t.id_turma, t.nome AS nome_turma, v.ano_letivo, b.numer
o AS numero_bimestre,
    tg.id_tag, tg.nome AS nome_tag, tg.cor AS cor_tag,
    COUNT(*) AS qtd_usos,
    ROUND(AVG(a.media), 2) AS media_dos_alunos
FROM ACOMPANHAMENTO a
```

```
JOIN ACOMPANHAMENTO_TAG atg ON atg.id_acompanhamento = a.id
_acompanhamento AND atg.ativo = 1
JOIN TAG tg ON tg.id_tag = atg.id_tag AND tg.ativo = 1
JOIN VINCULO_PROFESSOR v ON v.id_vinculo = a.id_vinculo
JOIN TURMA t ON t.id_turma = v.id_turma
JOIN BIMESTRE b ON b.id_bimestre = a.id_bimestre
WHERE a.ativo = 1 AND a.status = 'PUBLICADO'
GROUP BY
    t.id_turma, t.nome, v.ano_letivo, b.numero, tg.id_tag,
tg.nome, tg.cor;
```